

Privacy-First Hybrid ML Transaction Classifier

Team Name: Data Voyagers

Team Members:

1. B. Naga Akshaya
2. G. Deva Harshini
3. K. Kalanjali

GitHub Link:

https://github.com/Akshaya1307/GHCI_DataVoyagers_Hackerton

Live Demo Link:

<https://ghcidatavoyagershackerton-42f9p2bh3g6t74p2hwkqld.streamlit.app/>

Video Demo Link:

(To be added by team)

1. Introduction

In the modern financial ecosystem, digital transactions have become the dominant form of payment. With UPI, online banking, wallets, and credit systems expanding rapidly, categorizing transaction text into meaningful financial groups has become essential. Whether it's budgeting, analytics, tax preparation, or fraud detection, transaction categorization forms the core engine behind most fintech applications.

This project presents a fully working, privacy-first hybrid ML + rule-based transaction classifier. The system is designed to classify raw financial transaction text—such as “UPI AMAZON PAY 499,” “ZOMATO 299,” or “HP PETROL PUMP 1200”—into categories like Shopping, Food & Dining, Fuel, Wallet, Entertainment, EMI/Loan, and more.

Unlike cloud-dependent solutions, this system is built to run efficiently with local ML inference. It ensures privacy, scalability, and offline capability, making it ideal for fintech prototypes, personal finance applications, and standalone analytics tools.

2. Problem Statement

Raw financial transaction descriptions are inconsistent, unstructured, and often incomprehensible without context. Typical issues include alphanumeric strings, abbreviations, random numbers, multilingual vendor names, missing category indicators, and repeated inconsistencies between banks.

Example challenges:

- "AMZPYMT 499 GJ" → Shopping
- "HPCL 1200" → Fuel
- "NTR FRANCHISE ATM WDL" → ATM Withdrawal
- "ZMT* FOOD 230" → Food
- "G-PAY UPI REF1234" → Wallet/Transfer

Manual categorization is time-consuming and error-prone. Existing APIs are expensive and compromise privacy by sending user transaction data to third-party servers.

This project solves these problems using a **Hybrid ML + Rule Engine**.

3. System Objectives

The primary goals of this system include:

- 100% on-device processing
- Privacy-first architecture
- Accurate classification of mixed-format UPI and bank transaction text
- Hybrid model combining ML prediction with deterministic rules
- Simple and interactive UI using Streamlit
- Easy integration for fintech prototypes and analytics applications

4. System Architecture

The system combines two engines:

- A) Rule Engine
- B) Machine Learning Model (TF-IDF + Classifier)

High-Level Flow:

1. User pastes multiple transactions into the Streamlit interface.
2. Each line is cleaned, formatted, and preprocessed.
3. Rule engine checks for keyword matches (e.g., PETROL, HPCL → Fuel).
4. If no rule matches, ML model predicts the best-fit category.
5. Results are returned in a structured, table format.

This hybrid architecture offers:

- High accuracy
- Fast real-time inference
- Interpretability
- Reliability even on unseen entries

5. Dataset and Preprocessing

The dataset for this prototype consists of:

- Cleaned UPI text samples
- Manually labelled merchant categories
- Synthetic transaction patterns
- Common merchant keyword sets

Preprocessing includes:

- Lowercasing text
- Removing numbers and noise tokens
- Removing common words (PVT, LTD, STORE)
- Extracting merchant keywords
- TF-IDF vectorization

This prepares the text for ML classification.

6. Rule Engine Design

The rule engine detects strong keywords such as:

- PETROL / HPCL / HP → Fuel
- ZOMATO / SWIGGY → Food & Dining
- AMAZON / FLIPKART / BIGBAZAAR → Shopping
- NETFLIX / SPOTIFY → Subscription
- ATM / WITHDRAWAL → ATM
- EMI / LOAN → EMI/Loan

If a rule confidently detects a category, the ML model is skipped.

This reduces misclassifications and improves reliability.

7. Machine Learning Pipeline

The ML model uses:

- TF-IDF Vectorizer
- A trained classifier (such as Logistic Regression / Random Forest depending on training)

Pipeline:

1. Text input
2. TF-IDF vectorization
3. Model inference
4. Category output

The model is saved as a .pkl file and loaded during runtime.

8. Prototype Implementation (Streamlit App)

The application is built using Streamlit, offering:

- A clean UI
- A large multi-line text input box
- “Classify Transactions” button
- Table-based results
- Rule + ML output pipeline

The app link:

<https://ghcidatavoyagershackerton-42f9p2bh3g6t74p2hwkqld.streamlit.app/>

User Flow:

1. Paste multiple transactions
2. Click classify
3. View category results

9. Results

The hybrid engine offers:

- High accuracy for known patterns
- Robust classification for noisy UPI text
- Good generalization to unseen merchant names

Rule engine handles known merchants with 100% precision.

ML handles ambiguous entries more flexibly.

10. Security and Privacy

- The model runs entirely within the deployed Streamlit environment.
- No third-party API calls are used.
- User data is processed locally during the session.
- No logs store sensitive information.

11. Limitations

- Limited dataset size
- Challenges with completely new merchant names
- No probability scores displayed
- No multilingual support yet

12. Future Enhancements

- Add multilingual NLP support
- Train larger, more robust transformer-based models
- Implement category confidence scores
- Add expenditure visualization dashboards
- Add incremental learning
- Increase dataset size for better accuracy
- Add Fraud Detection and Pattern Mining

13. Conclusion

This project demonstrates a functioning, deployable, privacy-first Hybrid ML Transaction Categorizer. It balances accuracy and interpretability using rule-engine logic and ML classification, providing a lightweight and efficient solution for fintech applications.

With clean deployment, real-time inference, and an intuitive Streamlit frontend, the system serves as a strong foundation for future financial analytics systems.
